



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)

B. TECH

Year: 2026 – 27

Semester: VII

Name of the Student: Swayam Mandhani

PRN: 123B1B184

Department: Computer Engineering

Division: C, **Batch** – C2

Course: DevOps

Course Code: BCE27PE01

Assignment No – 6

Title:

Integrate Jenkins with a Git Repository and Automate Source Code Retrieval from GitHub.

Aim:

To configure Jenkins with a GitHub repository and automate source-code retrieval for a continuous integration workflow, followed by execution of a simple build using a Jenkins Freestyle project.

Objectives:

- Understand the role of Jenkins in a Continuous Integration / Continuous Delivery (CI/CD) workflow.
- Install and configure the Git, GitHub, and Pipeline-related Jenkins functionality required for the practical.
- Connect Jenkins to a GitHub repository using Git-based Source Code Management (SCM).
- Create and configure a Jenkins Freestyle job for automatic source-code checkout.
- Execute a simple build command on the retrieved source code and verify the build result.
- Demonstrate repeated source retrieval after changes are pushed to the GitHub repository.
- Configure SCM polling to support automated checking for repository changes.

Theory:

Continuous Integration and Jenkins

Continuous Integration (CI) is a development practice in which source-code changes are integrated frequently and automatically checked through build and test processes. Jenkins is an automation server

used to implement CI/CD workflows. It can retrieve source code from version-control systems, execute build or test commands, and report the result.

Jenkins and GitHub Integration

GitHub acts as the source-code repository while Jenkins acts as the automation server. Jenkins uses Git-based Source Code Management (SCM) configuration to connect to the repository, fetch changes, checkout the selected branch, and make the source code available in the Jenkins workspace.

Freestyle Project

A Jenkins Freestyle project provides a graphical job configuration model. Source Code Management, build triggers, build steps, and post-build actions can be configured through the job interface. For this assignment, a Freestyle project named GitHub-Jenkins-Demo was used.

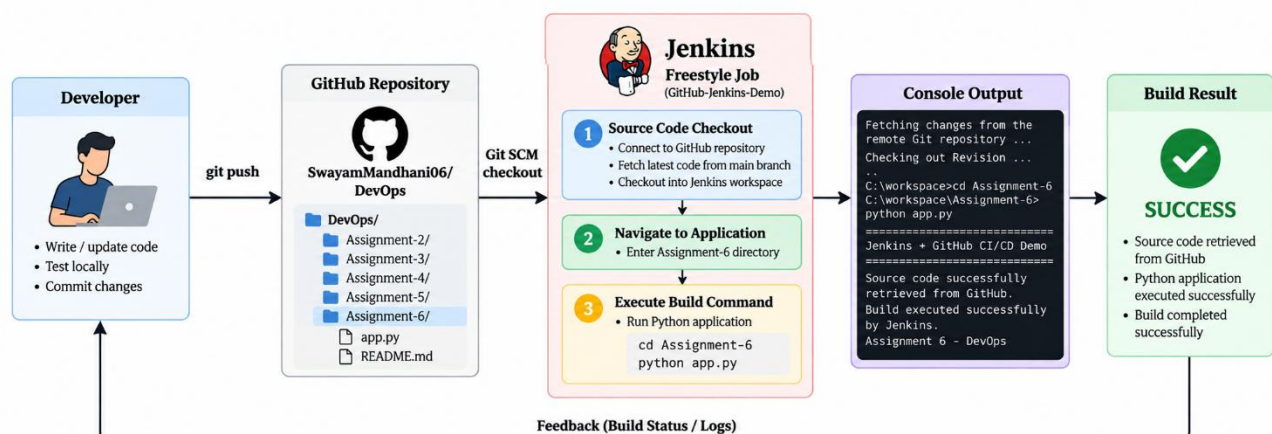
Automated Source Retrieval

Jenkins can retrieve source code when a job is executed. SCM polling can additionally be configured so Jenkins periodically checks the configured repository for changes. In this practical, Poll SCM was configured with the schedule H/5 * * * * after successful manual builds were verified.

Jenkins Workspace

The Jenkins workspace is the directory where the selected source repository is checked out for a build. The retrieved DevOps repository contains the Assignment-6 directory, from which the Python application is executed.

System / CI Workflow:



The implemented workflow uses the public DevOps repository at

<https://github.com/SwayamMandhani06/DevOps.git>.

The Jenkins job checks out the main branch and executes the application stored under Assignment-6.

Prerequisites and Tools:

- Jenkins LTS installed and running as a Windows service.
- Java 21 LTS configured for Jenkins.
- Git installed and available to Jenkins.
- GitHub account and public DevOps repository.
- Python 3.12 installed on the local Windows system.
- Web browser and PowerShell for configuration and verification.

Code / Implementation:

GitHub Repository Structure

DevOps/

|— Assignment-2/

|— Assignment-3/

|— Assignment-4/

|— Assignment-5/

|— Assignment-6/

| |— app.py

| |— README.md

|— README.md

Python Application — Assignment-6/app.py

```
print("=====")
```

```
print(" Jenkins + GitHub CI/CD Demo")
```

```
print("=====")
```

```
print("Updated source code successfully retrieved from GitHub.")
```

```
print("Jenkins executed the latest version from the main branch.")
```

```
print("Assignment 6 - DevOps")
```

Jenkins Job Configuration

Jenkins Freestyle project:

GitHub-Jenkins-Demo

Source Code Management:

SCM: Git

Repository URL: <https://github.com/SwayamMandhani06/DevOps.git>

Credentials: None (public repository)

Branch Specifier: */main

Build Step:

cd Assignment-6

"C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe" app.py

The absolute Python executable path was used because Jenkins runs as the Windows SYSTEM account and therefore does not automatically inherit the Python PATH configured for the normal user account.

Jenkins Pipeline

Pipeline as Code was implemented using a Jenkinsfile stored in the GitHub repository. The Jenkins Pipeline retrieves the source code from the main branch and executes three stages: Checkout, Build, and Test.

Jenkinsfile

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
            url: 'https://github.com/SwayamMandhani06/DevOps.git'
      }
    }
    stage('Build') {
      steps {
```

```
        echo 'Building application...'
        bat 'cd Assignment-6 &&
"C:\\Users\\Admin\\AppData\\Local\\Programs\\Python\\Python312\\python.exe" app.py'
    }
}
stage('Test') {
    steps {
        echo 'Running tests...'
    }
}
}
```

SCM Polling Configuration

Build Trigger:

Poll SCM

Schedule:

H/5 * * * *

The polling configuration enables Jenkins to periodically check the configured Git repository for changes. Manual Build Now execution was also used to verify the checkout and build workflow.

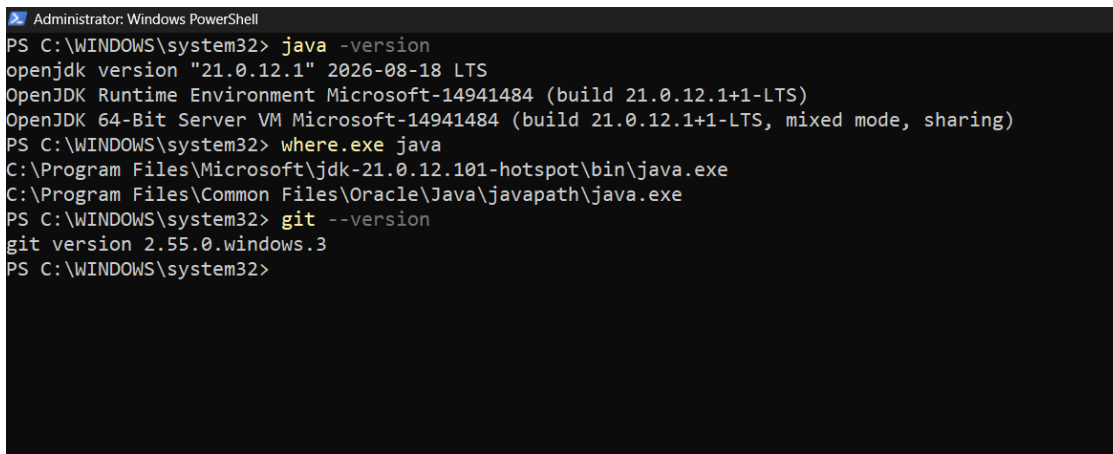
Implementation Steps

1. Installed Jenkins LTS and configured Java 21.
2. Completed the initial Jenkins setup and created the administrator account.
3. Verified Jenkins Git/GitHub-related functionality from the installed plugins.
4. Created the Assignment-6 folder in the existing DevOps GitHub repository.
5. Added app.py and README.md and pushed the files to the main branch.
6. Created the Jenkins Freestyle project GitHub-Jenkins-Demo.
7. Configured Git SCM with the DevOps repository URL and */main branch.
8. Configured the Windows batch build step to enter Assignment-6 and execute the Python application.
9. Executed Build #1; GitHub checkout succeeded, but the build initially failed because the Jenkins SYSTEM account could not resolve the python command.

10. Verified the installed Python 3.12 executable path and updated the Jenkins build step to use the absolute Python executable path.
11. Executed subsequent builds successfully and verified the application output and Finished: SUCCESS message.
12. Updated the source code, pushed the change to GitHub, and executed another Jenkins build to demonstrate retrieval of the updated source.
13. Configured Poll SCM with H/5 * * * * to automate periodic checking for repository changes.

Output

The following screenshots provide the major evidence for Jenkins installation, GitHub integration, source-code checkout, build execution, workspace retrieval, and repeated builds.



```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Microsoft-14941484 (build 21.0.12.1+1-LTS)
OpenJDK 64-Bit Server VM Microsoft-14941484 (build 21.0.12.1+1-LTS, mixed mode, sharing)
PS C:\WINDOWS\system32> where.exe java
C:\Program Files\Microsoft\jdk-21.0.12.101-hotspot\bin\java.exe
C:\Program Files\Common Files\Oracle\Java\javapath\java.exe
PS C:\WINDOWS\system32> git --version
git version 2.55.0.windows.3
PS C:\WINDOWS\system32>
```

Figure 1: Jenkins Prerequisites — Java 21 and Git Verification

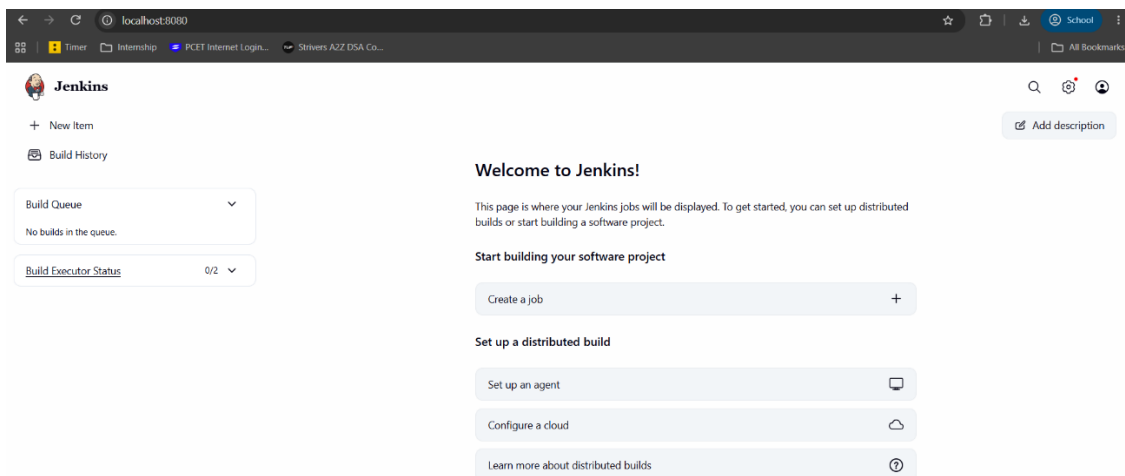


Figure 2: Jenkins Dashboard After Installation and Initial Configuration

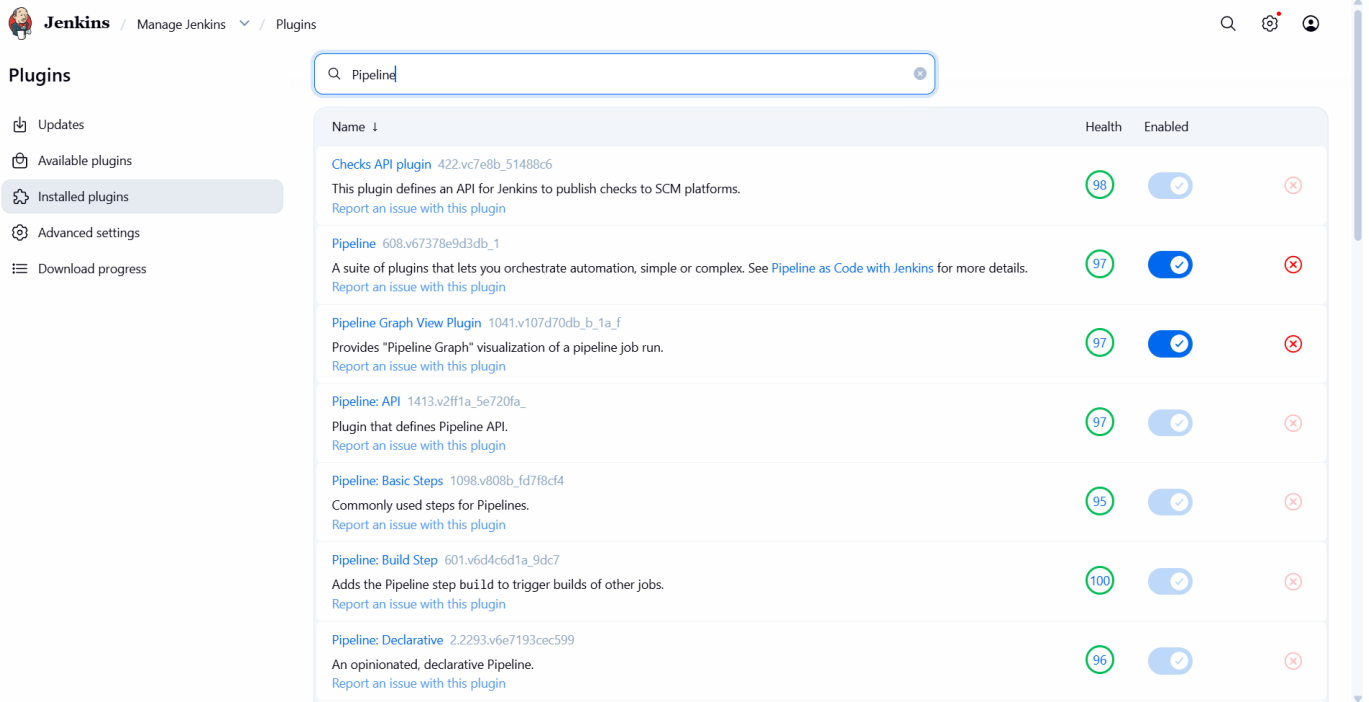


Figure 3: Jenkins Plugins Required for GitHub Integration

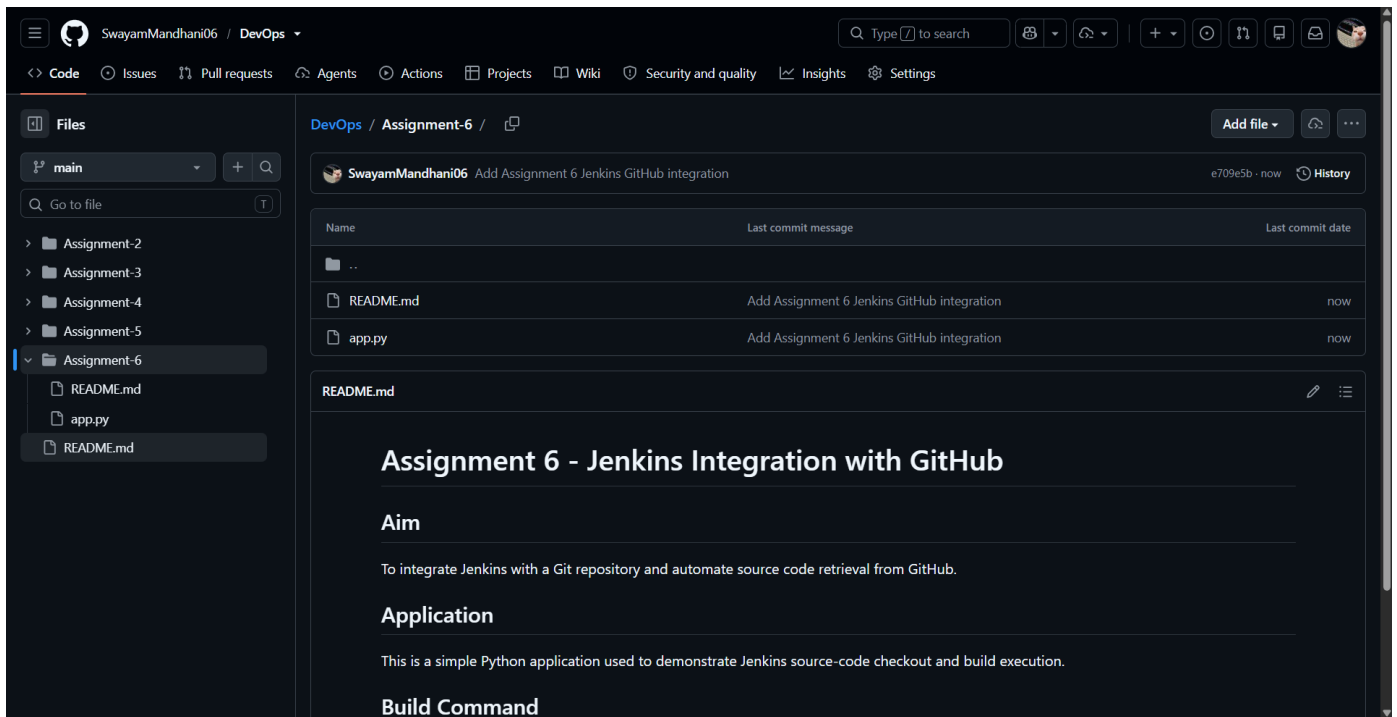
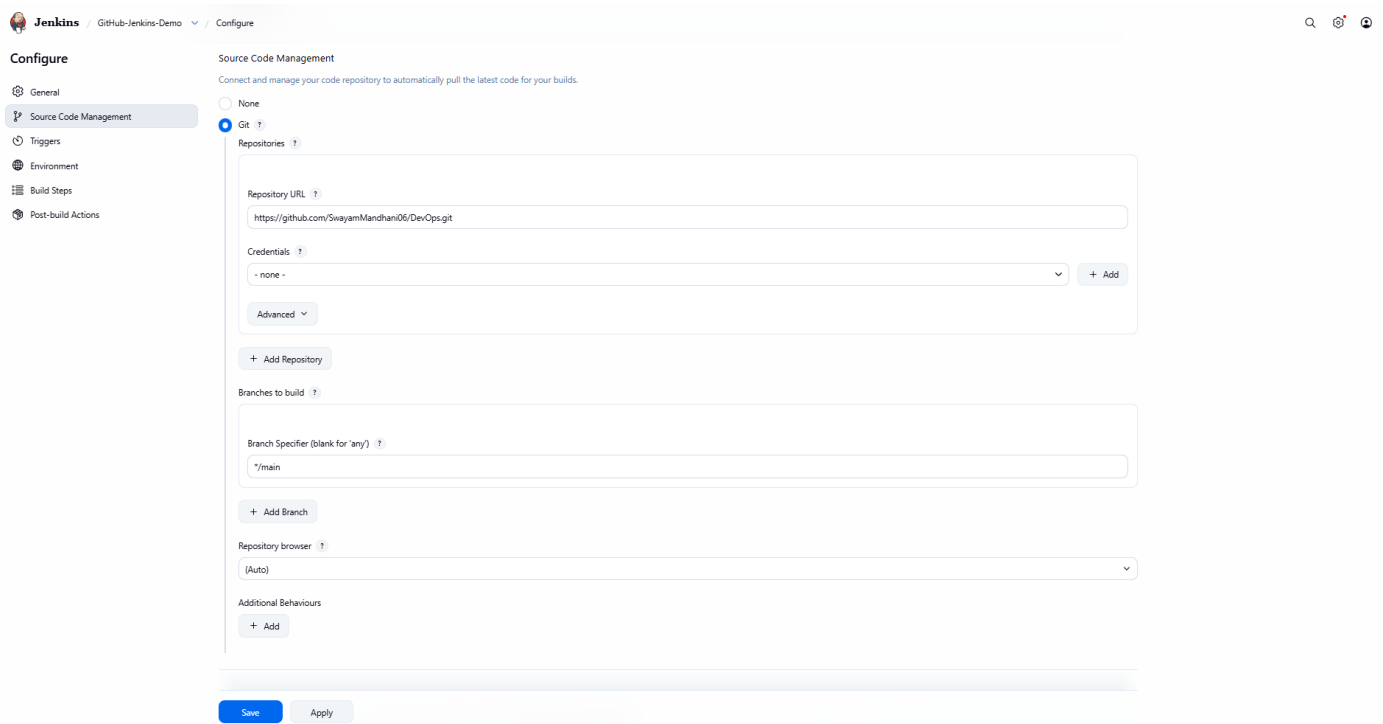


Figure 4: Assignment-6 Source Code Stored in GitHub Repository



The image shows the Jenkins configuration page for Source Code Management. The left sidebar contains a 'Configure' menu with options: General, Source Code Management (selected), Triggers, Environment, Build Steps, and Post-build Actions. The main content area is titled 'Source Code Management' and includes a description: 'Connect and manage your code repository to automatically pull the latest code for your builds.' Below this, there are radio buttons for 'None' and 'Git' (selected). The 'Git' section contains a 'Repositories' list with one entry: 'Repository URL' set to 'https://github.com/SwayamMandhani06/DevOps.git', 'Credentials' set to 'none', and an 'Advanced' dropdown. There is an '+ Add Repository' button. Below the repositories, there is a 'Branches to build' section with a 'Branch Specifier (blank for \'any\')' set to '*/main' and an '+ Add Branch' button. At the bottom, there is a 'Repository browser' dropdown set to '(Auto)' and an 'Additional Behaviours' section with an '+ Add' button. At the very bottom, there are 'Save' and 'Apply' buttons.

Jenkins / GitHub-Jenkins-Demo / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git

Repositories

Repository URL:

Credentials: + Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any'):

+ Add Branch

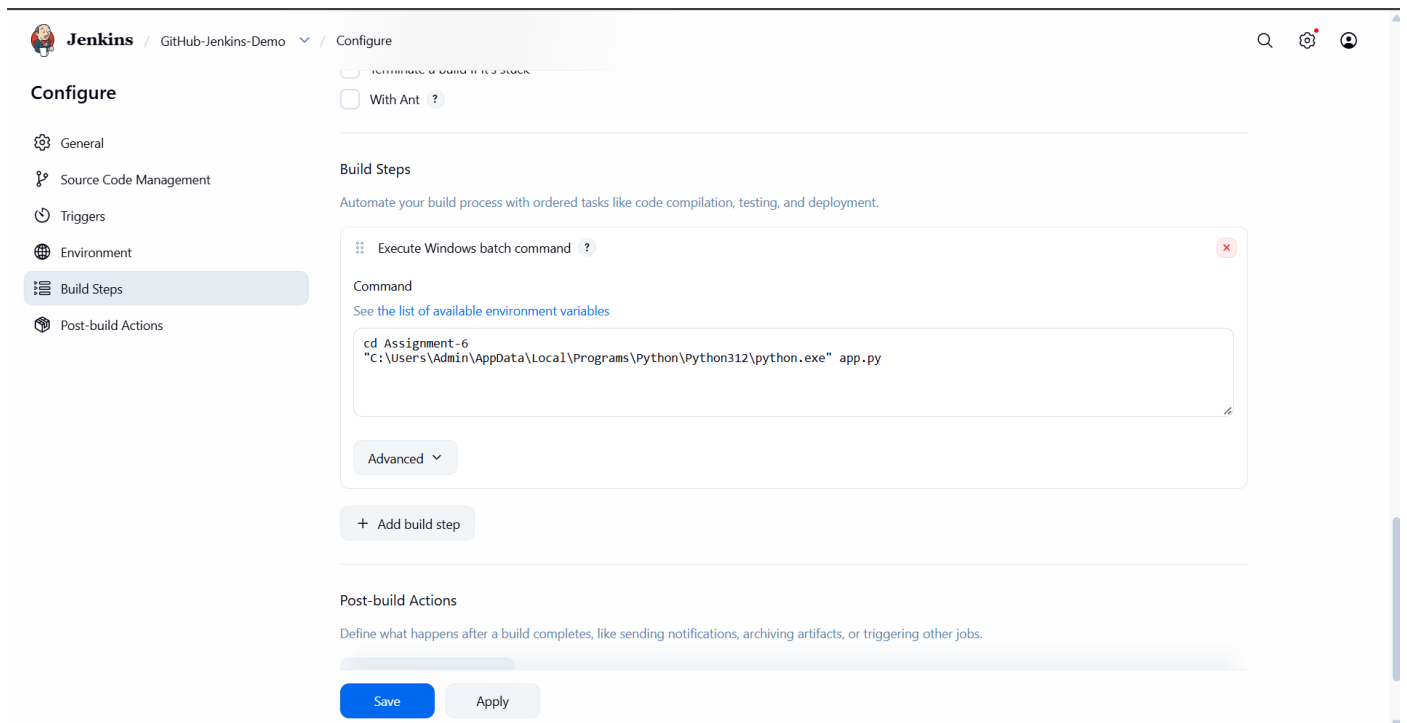
Repository browser:

Additional Behaviours

+ Add

Save Apply

Figure 5: Jenkins Source Code Management Configuration



The image shows the Jenkins configuration page for Build Steps. The left sidebar is the same as in Figure 5, with 'Build Steps' selected. The main content area is titled 'Build Steps' and includes a description: 'Automate your build process with ordered tasks like code compilation, testing, and deployment.' Below this, there is a 'Execute Windows batch command' step. The 'Command' field contains the text: 'cd Assignment-6' and '"C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe" app.py'. There is an 'Advanced' dropdown and an '+ Add build step' button. Below the build steps, there is a 'Post-build Actions' section with a description: 'Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.' At the bottom, there are 'Save' and 'Apply' buttons.

Jenkins / GitHub-Jenkins-Demo / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute Windows batch command

Command

See the list of available environment variables

cd Assignment-6
"C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe" app.py

Advanced

+ Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Save Apply

Figure 6: Jenkins Build Step Configuration



Figure 7: Jenkins Console Output Showing GitHub Checkout and Successful Build



Figure 8: Jenkins Workspace Containing Retrieved Source Code

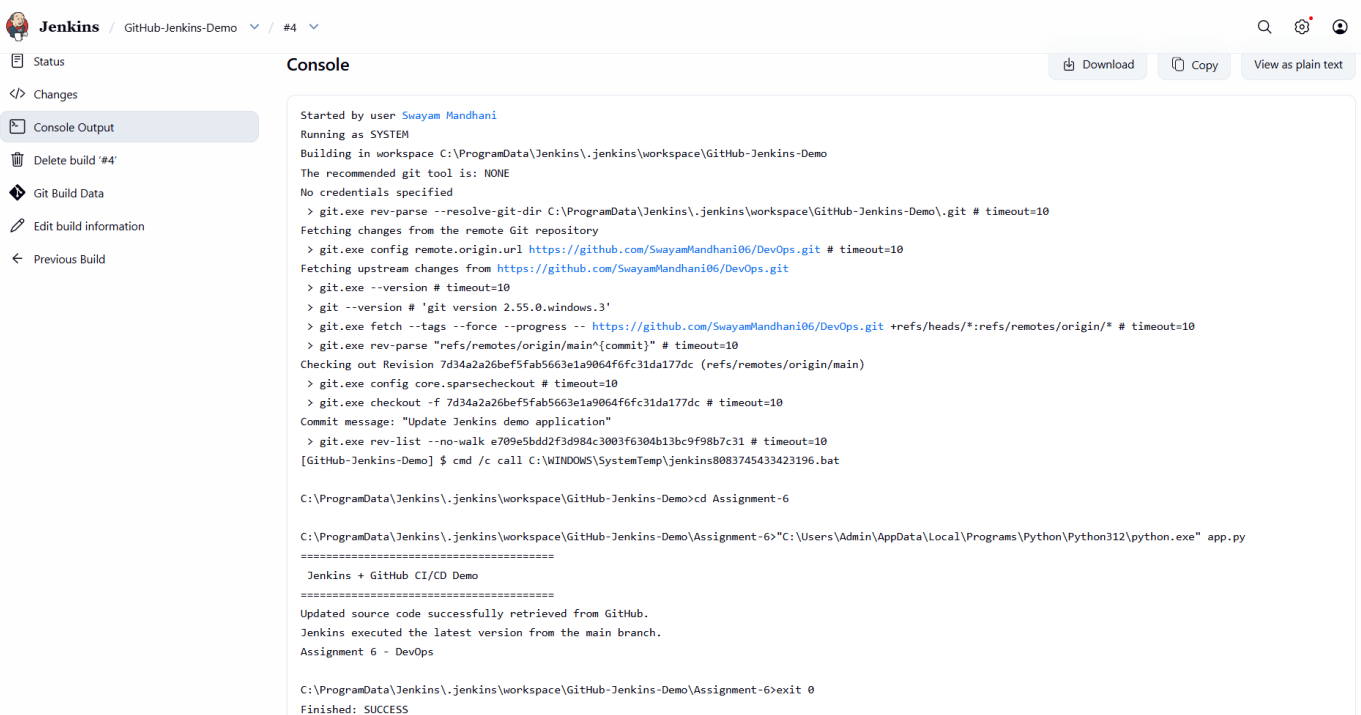


Figure 9: Jenkins Build After Updated GitHub Source

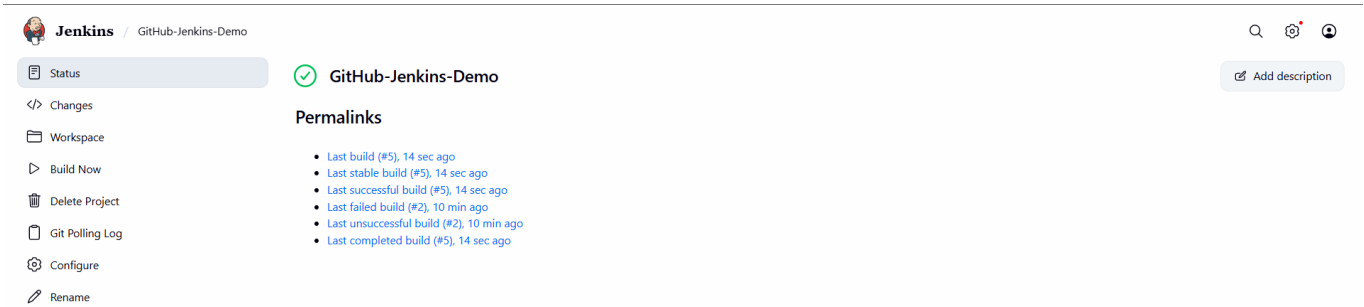


Figure 10: Jenkins Build History

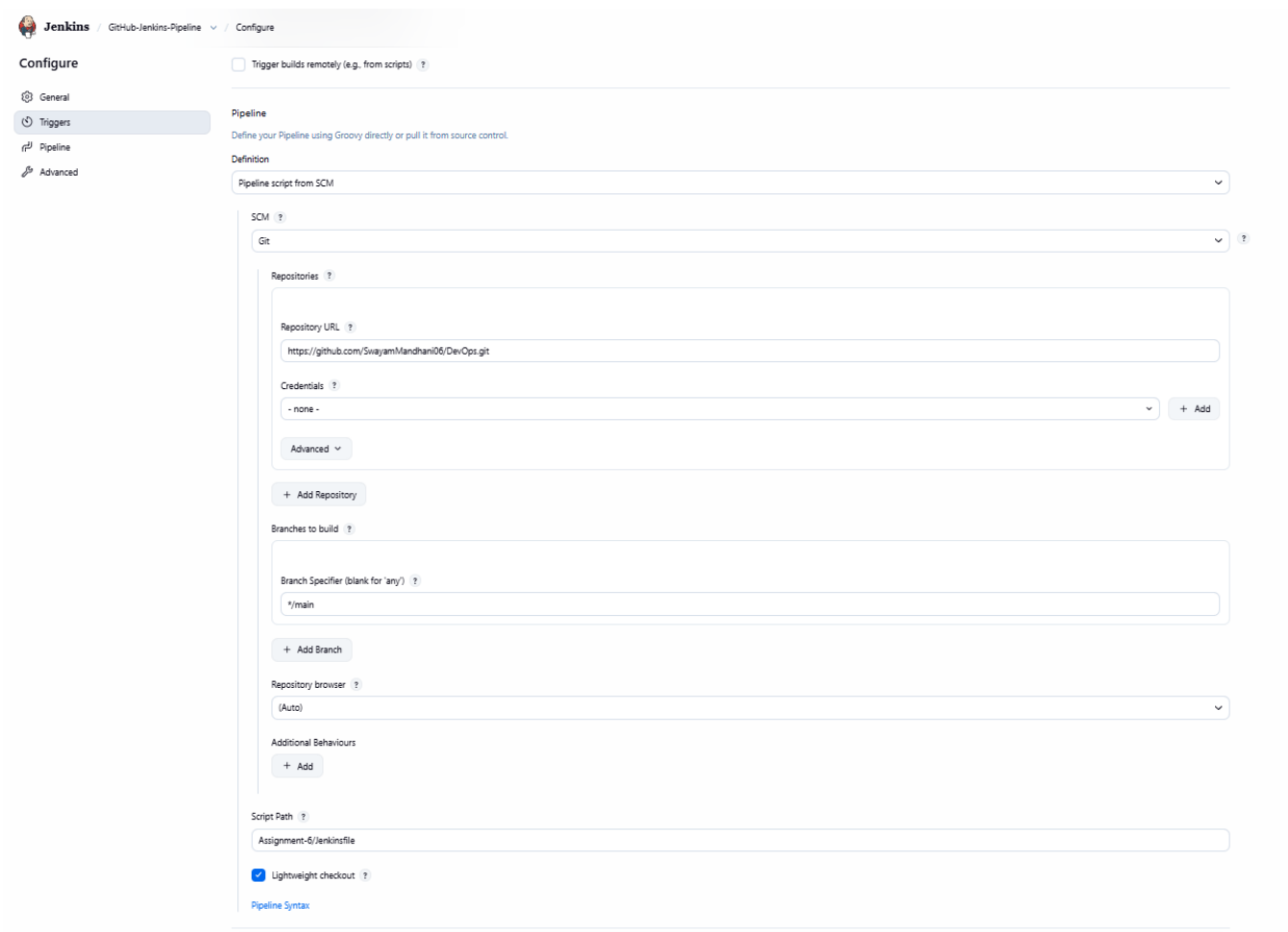


Figure 11: Jenkins Pipeline Configuration

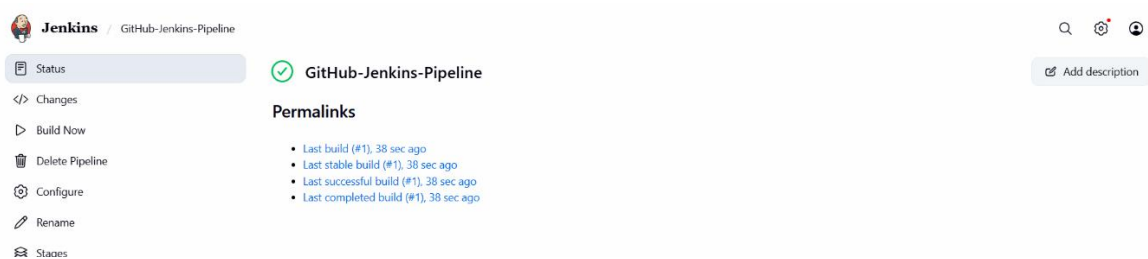


Figure 12: Jenkins Pipeline Build

The screenshot displays the Jenkins web interface for a pipeline named 'GitHub-Jenkins-Pipeline'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Delete build '#1', Git Build Data, Pipeline Overview, Edit build information, Restart from Stage, Replay, Pipeline Steps, and Workspaces. The main area shows the 'Console' output for build #1. The output text is as follows:

```
Started by user Swayam Mandhani
Obtained Assignment-6/Jenkinsfile from git https://github.com/SwayamMandhani06/DevOps.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\jenkins\workspace\GitHub-Jenkins-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git.exe
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/SwayamMandhani06/DevOps.git
> git.exe init C:\ProgramData\Jenkins\jenkins\workspace\GitHub-Jenkins-Pipeline # timeout=10
Fetching upstream changes from https://github.com/SwayamMandhani06/DevOps.git
> git.exe --version # timeout=10
> git --version # 'git version 2.55.0.windows.3'
> git.exe fetch --tags --force --progress -- https://github.com/SwayamMandhani06/DevOps.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git.exe config remote.origin.url https://github.com/SwayamMandhani06/DevOps.git # timeout=10
> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision b1f96dc5f23a6802ed3da0f06e0a0182f03be8f9 (refs/remotes/origin/main)
> git.exe config core.sparsecheckout # timeout=10
> git.exe checkout -f b1f96dc5f23a6802ed3da0f06e0a0182f03be8f9 # timeout=10
Commit message: "Added Jenkinsfile for pipeline"
First time build. Skipping changelog.
[Pipeline] }
```

Figure 13: Jenkins Pipeline Console Output

Result:

Jenkins was successfully integrated with the GitHub repository

<https://github.com/SwayamMandhani06/DevOps.git>.

A Freestyle project named GitHub-Jenkins-Demo was configured to retrieve source code from the main branch. Jenkins successfully checked out the repository, entered the Assignment-6 directory, executed the Python application, and reported a successful build. A subsequent source-code update was pushed to GitHub and successfully retrieved and executed by Jenkins. SCM polling was also configured to support automated checking for repository changes. Additionally, a Jenkins Pipeline was implemented using a Jenkinsfile stored in the GitHub repository, with Checkout, Build, and Test stages executed successfully.

Conclusion:

In this assignment, I learned how Jenkins supports Continuous Integration by connecting an automation server with a Git repository. I configured Jenkins with GitHub, created a Freestyle job, configured Git-based Source Code Management, checked out the main branch, and executed a build step on the retrieved source code. I also resolved a Windows service-environment PATH issue by explicitly specifying the Python executable used by Jenkins. Finally, I demonstrated repeated source retrieval

after a GitHub update and configured SCM polling for automated repository checking. This practical provided hands-on understanding of the GitHub -> Jenkins -> Checkout -> Build -> Result workflow. The Pipeline implementation further demonstrated Pipeline-as-Code and stage-based CI workflow execution using Jenkins.

References:

1. Jenkins Documentation — <https://www.jenkins.io/doc/>
2. Jenkins User Documentation — Installing Jenkins on Windows — <https://www.jenkins.io/doc/book/installing/windows/>
3. Jenkins User Documentation — Managing Jenkins — <https://www.jenkins.io/doc/book/managing/>
4. Git Documentation — <https://git-scm.com/doc>
5. GitHub Documentation — <https://docs.github.com/>

Submission Links:

GitHub Repository Link: <https://github.com/SwayamMandhani06/DevOps/tree/main/Assignment-6>

Hosted / Output Link: NA – Jenkins practical executed locally at <http://localhost:8080>